

Hunter in Maze

3인칭 FPS 게임 프로젝트

 신동휘

 윤태웅

 Unreal Engine 5



01. Game Introduction

게임 소개

MISSION BRIEF

제한 시간 내에 무작위로 생성되는 미로 안에서
최대한 많은 적을 사냥하여 점수를 획득하는
3인칭 생존 FPS 게임입니다.

CORE FEATURES

매번 변화하는 절차적 미로 생성

거리 기반 지능형 적 AI (외위)

진정감 넘치는 타일 어택 시스템

SURVEILLANCE_FEED_01



02. Development Goals

개발 목표

초기 기획 단계에서는
더 방대하고 디테일한 시스템을
목표로 했습니다.

REALITY CHECK

제한된 개발 기간과 인력으로 인해,
"핵심 재미(Core Fun)" 구현에 집중하기로 결정했습니다.



FOCUSED ON IMPLEMENTATION



EXTENDED GOALS (Wishlist)



Advanced Ammo System

DEFERRED

현실적인 재장전 애니메이션, 탄창 관리 로직



Enemy Variety & Scoring

DEFERRED

엘리트 적, 적 유형별 작동 점수 시스템



Modular Prefab Swapping

DEFERRED

언타일 맵 테마 및 무기 프리팹 교체 구조



Polished UI/UX

DEFERRED

고도화된 HUD 디자인 및 연출 효과

* These features are planned for future updates (v2.0)

03. Implemented Features

구현된 기능

Maze Generator

게임 시작 시마다 새로운 구조의 미로를 생성하는 절차적 생성 시스템 구현.

- > Algorithm: Recursive Backtracking (DFS)
- > Runtime Generation

Enemy AI

플레이어와의 거리를 실시간으로 계산하여 회피(Flee) 행동 전환.

- > Logic: Distance-based State Machine
- > Optimization: Squared Distance Check

Rapid Fire Weapon

마우스 클릭 유지 시 연속 발사가 가능한 연사(Full-Auto) 기능 구현.

- > Input: KeyDown / KeyUp Event Handling
- > Fire Rate Control

Time Limit System

제한 시간 카운트다운 및 시간 종료 시 게임 오버 처리 로직 구현.

- > Manager: GameState, Timer
- > UI: Countdown, Win/Lose Display

05. Maze Generation System(cpp)

미로 생성 시스템

```
// 생성자
AMazeGenerator::AMazeGenerator()
{
    // Tick을 사용하지 않음 (미로는 시작 시 한 번만 생성)
    PrimaryActorTick.bCanEverTick = false;
}

// 게임 시작 시 호출
void AMazeGenerator::BeginPlay()
{
    Super::BeginPlay();

    // 미로 배열 초기화
    InitMaze();

    // (1,1) 위치부터 미로 생성 시작 (외곽 벽 보존)
    GenerateMaze(1, 1);

    // 생성된 미로 데이터를 기반으로 액터 배치
    SpawnMaze();
}
```

```
// 미로 배열 초기화 함수
void AMazeGenerator::InitMaze()
{
    // 세로 크기만큼 2차원 배열 생성
    Maze.SetNum(Height);

    for (int32 y = 0; y < Height; y++)
    {
        // 가로 크기 설정
        Maze[y].SetNum(Width);

        for (int32 x = 0; x < Width; x++)
        {
            // false = 벽, true = 길
            // 처음에는 전부 벽으로 초기화
            Maze[y][x] = false;
        }
    }
}
```

```

// DFS(깊이 우선 탐색)를 이용한 미로 생성 함수
void AMazeGenerator::GenerateMaze(int32 X, int32 Y)
{
    // 현재 위치를 길로 설정
    Maze[Y][X] = true;

    // 이동 방향 (상, 하, 좌, 우)
    TArray<FIntPoint> Directions = {
        {1, 0}, {-1, 0}, {0, 1}, {0, -1}
    };

    // 방향 배열을 랜덤으로 섞어서
    // 미로가 매번 다르게 생성되도록 함
    for (int32 i = 0; i < Directions.Num(); ++i)
    {
        int32 RandIndex = FMath::RandRange(0, Directions.Num() - 1);
        Directions.Swap(i, RandIndex);
    }

    // 모든 방향에 대해 탐색
    for (auto D : Directions)
    {
        // 두 칸 앞을 기준으로 다음 셀 계산
        int32 NX = X + D.X * 2;
        int32 NY = Y + D.Y * 2;

        // 미로 범위 체크 (외곽 벽 유지)
        if (NX > 0 && NX < Width - 1 && NY > 0 && NY < Height - 1)
        {
            // 아직 방문하지 않은 셀이라면
            if (!Maze[NY][NX])
            {
                // 현재 위치와 다음 위치 사이의 벽을 길로 변경
                Maze[Y + D.Y][X + D.X] = true;

                // 다음 위치에서 재귀적으로 미로 생성
                GenerateMaze(NX, NY);
            }
        }
    }
}

```

```

// 미로 데이터를 실제 액터로 배치하는 함수
void AMazeGenerator::SpawnMaze()
{
    UWorld* World = GetWorld();
    if (!World) return;

    // 미로 시작 위치
    FVector Origin = GetActorLocation();

    for (int32 y = 0; y < Height; y++)
    {
        for (int32 x = 0; x < Width; x++)
        {
            // 셀 위치 계산
            FVector Pos = Origin + FVector(x * CellSize, y * CellSize, 0.f);

            if (Maze[y][x]) // 길인 경우
            {
                if (FloorClass)
                {
                    // 바닥 액터 생성
                    World->SpawnActor<AActor>(FloorClass, Pos, FRotator::ZeroRotator);
                }
            }
            else // 벽인 경우
            {
                if (WallClass)
                {
                    // 벽 액터 생성
                    World->SpawnActor<AActor>(WallClass, Pos, FRotator::ZeroRotator);
                }
            }
        }
    }
}

```

06. EnemyFSM(cpp)

미로 생성 시스템

```
void UEnemyFSM::MoveState()
{
    // 플레이어 위치
    FVector playerPos = target->GetActorLocation();

    // 적(자기 자신) 위치
    FVector myPos = me->GetActorLocation();

    /* ===== 도망 방향 계산 ===== */

    // 플레이어 → 적 방향의 반대 벡터
    // 즉, 플레이어로부터 멀어지는 방향
    FVector fleeDir = (myPos - playerPos).GetSafeNormal();

    // 도망갈 거리 (값이 클수록 더 멀리 도망감)
    float fleeDistance = 100.0f;
```

```
// 도망 목표 위치 = 현재 위치 + 도망 방향 * 거리
FVector fleePos = myPos + fleeDir * fleeDistance;

/* ===== NavMesh 위의 유효한 위치인지 확인 ===== */

// 네비게이션 시스템 가져오기
auto ns = UNavigationSystemV1::GetNavigationSystem(GetWorld());

FNavLocation navLoc;

// 계산된 도망 위치가 NavMesh 위에 있는지 검사
bool valid = ns->ProjectPointToNavigation(fleePos, navLoc);
```


06. EnemyFSM(cpp)

미로 생성 시스템

```
if (valid)
{
    // NavMesh 위라면 해당 위치로 이동
    ai->MoveToLocation(navLoc.Location, 30.0f);
}
else
{
    // NavMesh 위가 아니라면
    // 반경 내에서 랜덤한 도망 위치를 다시 탐색
    GetRandomPositionInNavMesh(myPos, 800, fleePos);

    // 랜덤으로 찾은 위치로 이동
    ai->MoveToLocation(fleePos);
}
```

```
/* ===== 상태 전환 조건 ===== */

// 플레이어와의 현재 거리 계산
float distance = FVector::Distance(playerPos, myPos);

// 충분히 멀어졌다면 도망 상태 종료
if (distance > 1001.0f)
{
    // Idle 상태로 전환
    mState = EEnemyState::Idle;

    // 애니메이션 상태도 함께 변경
    anim->animState = mState;
}
```

07. PlayerFire(cpp)

미로 생성 시스템

```
void UPlayerFire::SetupInputBinding(UEnhancedInputComponent* PlayerInputComponent)
{
    //Started에서 Triggered로 변경하여 연사 가능하게 수정
    PlayerInputComponent->BindAction(iaFire, ETriggerEvent::Triggered, this, &UPlayerFire::InputFire);
    PlayerInputComponent->BindAction(iaGrenadeGun, ETriggerEvent::Started, this, &UPlayerFire::ChangeToGrenadeGun);
    PlayerInputComponent->BindAction(iaRifleGun, ETriggerEvent::Started, this, &UPlayerFire::ChangeToRifleGun);
    PlayerInputComponent->BindAction(iaSniper, ETriggerEvent::Started, this, &UPlayerFire::SniperAim);
    PlayerInputComponent->BindAction(iaSniper, ETriggerEvent::Completed, this, &UPlayerFire::SniperAim);
}

void UPlayerFire::InputFire(const struct FInputActionValue& inputValue)
{
    float now = GetWorld()->GetTimeSeconds();

    // 쿨타임 체크
    if (now - lastFireTime < fireCooldown)
    {
        return; // 아직 쿨타임 안 됨
    }

    lastFireTime = now; // 발사 시간 갱신
```

07. PlayerFire(h)

미로 생성 시스템

```
// 0.1초 연사 쿨타임  
UPROPERTY(EditAnywhere, Category = Fire)  
float fireCooldown = 0.1f;
```

08. TPSCharacter(cpp)

미로 생성 시스템

```
void ATPSCharacter::Tick(float DeltaTime)
```

```
{
```

```
    Super::Tick(DeltaTime);
```

```
    // hpTimer는 실제 HP가 아니라
```

```
    // '시간 경과를 측정하기 위한 타이머 변수'로 사용
```

```
    hpTimer += DeltaTime;
```

```
    // 1초가 지났을 경우
```

```
    if (hpTimer >= 1.0f)
```

```
{
```

```
        // 매 1초마다 HP를 1씩 감소
```

```
        hp--;
```

```
        // 타이머 초기화 (다음 1초 측정을 위해)
```

```
        hpTimer = 0.0f;
```

```
        PRINT_LOG(TEXT("HP decreased: %d"), hp);
```

```
        // HP가 0 이하가 되면 게임 오버 처리
```

```
        if (hp <= 0)
```

```
{
```

```
            PRINT_LOG(TEXT("Game Over"));
```

```
            OnGameOver();
```

```
}
```

```
}
```

```
}
```

09. Key Technologies

핵심 기술



Procedural Maze Generation

던타형에 미로 구조를 동적으로 생성하는 알고리즘
매 게임마다 새로운 레이아웃을 제공하여 리플레이
감상

DFS Algorithm



Adaptive Enemy AI

플레이어와의 거리를 실시간으로 계산하여 행동 패턴
변화 (flee) 상태 전환 로직 적용

State Machine



Real-time System Management

타이머, 점수, 적 소문을 관리하는 게임 매니저
내장 게임 로직의 효율적인 데이터 바인딩 및

Game Engine

Optimization



10. Team Roles

역할 분담



신동휘



Enemy AI System

거리 기반 상태 전환 및 행동 로직 구현



UI Design & Logic

HUD 및 메뉴 시스템 설계



Maze Algorithm

미로 생성 알고리즘 개발



Maze Generation System

절차적 생성 시스템 구현 및 최적화



윤태웅



UI Integration

게임 로직과 UI 데이터 연동



Ammo & Weapon System

탄약관리 및 무기 교체 시스템 개발



COLLABORATION PROTOCOL

Modular Development

기능별 독립 모듈 개발로 통합 효율화

Integration Testing

모듈 통합 테스트를 통한 버그 예방

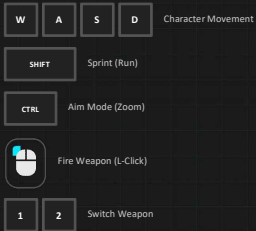
Version Control (Git)

이력 추적 및 협업 환경 관리

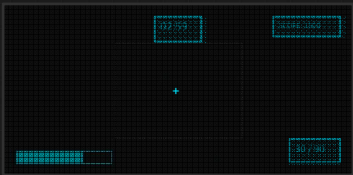
11. Controls & Interface

조작법 및 인터페이스

MOVEMENT & ACTION



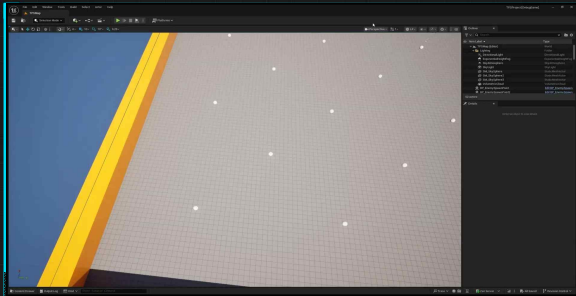
HEADS-UP DISPLAY (HUD)



- Timer:** Remaining time for the session
- Score:** Current points accumulated
- Ammo:** Current Mag / Total Reserve
- Health:** Player vitality status

12. Gameplay Demo

실행 영상



✔ Maze Generation

✔ Enemy Tracking

✔ Combat Action